

Packaging software for the Debian based systems

One of the most important things that you need to learn is to package and publish the code to make it available for the end users. Lets try and do this in a step by step manner as part of our simple tutorial.

Before you begin with creating packages, make sure that the following packages are installed on your system to enable you to use the utilities related to creating a deb package: -

- build-essential
- devscripts
- debhelper
- lintian

In order to understand the concept of creating packages, we need to learn that creating a package is mainly a three step process: -

- *Procuring the source tarball* : The source tarball is the compressed tar file of the source code of the application to be packaged, eg, `packagename.tar.gz`
- *Creating the Source package* : This step involves adding Debian specific packaging files and any modifications that you want to make to the package. This is done under a special directory named 'debian'.
- *Building the package*: Once the source package is ready, it can be built to create a binary package which can then be finally installed on the system.

In the first step, the source tarball needs to be renamed specially according to the accepted standards, mainly the fact that it needs to be in lowercase characters and in the format like: -

```
mv packagename-1.0.0.tar,gz packagename_1.0.0.orig.  
tar.gz
```

We can now unpack the source tarball and create the debian subdirectory under the source tree. Inside this directory following files are necessary to be provided: -

- *changelog* : This file summarizes the list of changes in this particular version of the package as compared to the previous version or mention that it is an initial release in case it is being packaged for the first time. This file can be easily created using the `dch` command as follows: -

```
dch -create -v 1.0.0 --package packagename
```

```

pkginstall-studio-1555-$ $pkg -s xterm
Package: xterm
Status: install ok installed
Priority: optional
Section: x11
Installed-Size: 1390
Maintainer: Ubuntu X-Swat <ubuntu-x@lists.ubuntu.com>
Architecture: i386
Multi-Arch: foreign
Version: 274-1ubuntu2
Provides: x-terminal-emulator
Depends: libtinfo, libnc (>= 2.15), libfontconfig (>= 2.6.0), libxkb (>= 1:1.0
+), libfribidi, libutempter (>= 1:1.5), libxft (>= 2.1.1),
libxmu, libxkb
Recommends: x11-utils
Suggests: xfonts-cyrillic
Conflicts:
/etc/X11/app-defaults/XUTerm-color: 440eb4b83bc79065c7ac85f26fca5
/etc/X11/app-defaults/XUTerm 42nd406a339b3b74a240415d1a51254
/etc/X11/app-defaults/XUTerm 5f9811c08fba9ac9b7dbd3d5cd0a5
/etc/X11/app-defaults/KOI8R/XUTerm-color: d2d262787629508b192169450281cf0
/etc/X11/app-defaults/XUTerm-color: 0eacc08b2f3c3be29edd064394496
/etc/X11/app-defaults/KOI8R/XUTerm 49040657376038273aaf940051316f
Description: X terminal emulator
xterm is a terminal emulator for the X window system. It provides VTE V102
and Tektronix 4014 compatible terminals for programs that cannot use the
window system directly. This version implements ISO/ANSI colors and most of
the control sequences used by DEC VT220 terminals.

This package provides four commands: xterm, which is the traditional
terminal emulator; xterm, which is a wrapper around xterm that is
intelligent about locale settings (especially those which use the UTF-8
character encoding), but which requires the full program from the x11-utils
package; koi8xterm, a wrapper similar to xterm for locales that use the
KOI8-R character set; and lxterm, a simple wrapper that chooses which of
the previous commands to execute based on the user's locale settings.

A complete list of control sequences supported by the X terminal emulator
is provided in /usr/share/doc/xterm.

The xterm program uses shared libraries provided by the libtinfo package.

```

Dpkg showing metadata about a package in Ubuntu

This will create a changelog file with a format as follows: -

```
vlc (2.0.8) UNRELEASED; urgency=low
* Initial release. (Closes: #XXXXXX)
```

```
-- Ankit Mathur <ankit@ankit-laptop> Sun, 15 Dec
2013 19:10:31 +0530
```

- *compat* : This file specifies the compatibility level of the application for debhelper, which contains a single digit number like 9 or 8 as its contents.
- *control* : This file provides information about the source and binary package, mainly details about the maintainer, priority, section, dependencies, description etc. A sample control file looks as follows: -

Source: helloworld

Section: unknown

Priority: extra

Maintainer: Ankit Mathur <ankit@unknown>

```
Build-Depends: debhelper (>= 8.0.0)
```

Standards-Version: 3.9.4

Homepage: <insert the upstream URL, if relevant>

```
#Vcs-Git: git://git.debian.org/collab-maint/hell-
loworld.git
```

```
#Vcs-Browser: http://git.debian.org/?p=collab-  
maint/helloworld.git;a=summary
```

```
Package: helloworld
```

Architecture: any

```
Depends: ${shlibs:Depends}, ${misc:Depends}
```

Description: <insert up to 60 chars description>
<insert long description, indented with spaces>

- *copyright* : This file keeps track of legal and copyright information.
- *rules* : This file acts as the makefile and consists of rules each of which specifies a target and how it is executed. This target specifies the various build targets which can be overridden when executed in the debhelper sequence. The most basic file looks like this: -

```
#!/usr/bin/make -f
```

```
# *- makefile -*-
```

```
# Sample debian/rules that uses debhelper.
```

```
# This file was originally written by Joey Hess
and Craig Small.
```

```
# As a special exception, when this file is copied
# by dh-make into a
```

```
# dh-make output file, you may use that output file
# without restriction.
```

```
# This special exception was added by Craig Small
in version 0.37 of dh-make.
```

```
# Uncomment this to turn on verbose mode.
```

```
#export DH VERBOSE=1
```

%

dh \$@

- *source/format*: This file contains the version information about the format of the source package, for eg. 3.0 (quilt) in this case.

There are a couple of other files that might be needed for completing the package and most of this process involves writing rules and ensuring that build process works correctly as expected, but once all that is done, the package can be built as follows: -